

Project Synopsis

For

CampusCompute: Campus-Aware Cloud Resource Pooling System

August 17, 2026

Prepared by

Specialization	SAP ID	Name
Computer Science	XXXXXXXXXX	Member 1 (Backend)
Computer Science	XXXXXXXXXX	Member 2 (Scheduling)
Computer Science	XXXXXXXXXX	Member 3 (Infrastructure)
Computer Science	XXXXXXXXXX	Member 4 (Full-Stack)

School Of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES,

DEHRADUN- 248007. Uttarakhand

TABLE OF CONTENTS

No.	Topic	Page No
	Table of Content	1
	Revision History	2
1	Introduction	3
1.1	Purpose of the Project	3
1.2	Target Beneficiary	3
1.3	Project Scope	4
1.4	References	5
2	Project Description	6
2.1	Reference Algorithm	6
2.2	Characteristic of Data	7
2.3	SWOT Analysis	9
2.4	Project Features	10
2.5	User Classes and Characteristics	11
2.6	Design and Implementation Constraints	13
2.7	Design Diagrams	15
2.8	Assumption and Dependencies	16
3	System Requirements	18
3.1	User Interface	18
3.2	Software Interface	20
3.3	Database Interface	22
3.4	Protocols	23
4	Non-functional Requirements	26
4.1	Performance Requirements	26
4.2	Security Requirements	27
4.3	Software Quality Attributes	29
5	Other Requirements	33
App. A	Glossary	35
App. B	Analysis Model	37
App. C	Issues List	39

REVISION HISTORY

Date	Change	Reason for Changes	Mentor Signature

1 INTRODUCTION

1.1 Purpose of the Project

College computer laboratories represent a significant capital investment, yet they experience dramatic utilization patterns. During scheduled lab sessions, machines operate at 80-100% capacity, while outside these hours—evenings, weekends, and holidays—utilization drops to as low as 5-20%. This represents substantial wasted computing capacity that could benefit students requiring computational resources for coursework, projects, and research.

The purpose of CampusCompute is to transform underutilized campus lab computers into a shared cloud computing platform. By intelligently pooling idle computational resources and providing on-demand access through isolated Docker containers, the system maximizes the return on investment of existing infrastructure while democratizing access to computing resources.

The project addresses three key challenges:

Resource Underutilization: Converting idle lab machines into productive assets during off-peak hours.

Access Inequality: Providing students without personal high-performance computers equal access to computational resources. **Academic Integration:** Respecting institutional schedules and lab reservations while optimizing resource allocation.

The motivation for this project stems from the observation that educational institutions maintain expensive computing infrastructure that remains idle for significant periods, while students often lack adequate computing resources for demanding academic work. CampusCompute bridges this gap through a campus-aware resource scheduling system.

1.2 Target Beneficiary

The prime beneficiaries of CampusCompute are:

Students: Undergraduate and graduate students gain on-demand access to isolated development environments from any location. Students working on resource-intensive projects—machine learning, data analysis, software development, simulation—can request computing resources without purchasing personal hardware.

Faculty: Instructors can allocate computational resources for assignments, projects, and research without managing individual machine access. Faculty conducting research can leverage pooled resources for computational experiments.

IT Administrators: Centralized management of computing resources across multiple labs. Improved resource utilization metrics and reduced pressure to purchase additional hardware.

Institution: Maximized return on infrastructure investment. Enhanced student satisfaction

and academic outcomes. Competitive advantage in attracting students through modern computing facilities.

1.3 Project Scope

CampusCompute is a distributed cloud resource management system designed specifically for educational institutions. The system aggregates computational resources from campus lab computers and provisions isolated containerized environments to authenticated users.

Application Area: Educational cloud computing, resource pooling, distributed systems, container orchestration.

Key Objectives:

- Develop a three-layer architecture: web interface, central broker, and distributed agents
- Implement secure authentication and role-based access control
- Create an adaptive scheduler that respects academic schedules and lab reservations
- Provide browser-based terminal access to containerized environments
- Enforce resource quotas and usage policies
- Monitor system health and resource utilization
- Ensure container isolation and security

Deliverables:

1. React-based web application (student and admin portals)
2. Spring Boot central broker with RESTful APIs
3. Python agent software for lab computers
4. PostgreSQL database schema and migrations
5. Docker container security policies
6. System documentation and deployment guides
7. Performance evaluation and research findings

Benefits:

- Increased lab utilization from approximately 18% to 50-70%

- Reduced need for additional hardware purchases
- Equitable access to computing resources
- Support for resource-intensive academic work
- Privacy-preserving resource sharing model

1.4 References

1. Docker Documentation: <https://docs.docker.com/>
2. Spring Boot Reference Guide: <https://spring.io/projects/spring-boot>
3. React Documentation: <https://react.dev/>
4. PostgreSQL Documentation: <https://www.postgresql.org/docs/>
5. Kubernetes Documentation: <https://kubernetes.io/docs/>
6. Burns, B., et al. (2016). *Borg, Omega, and Kubernetes*. ACM Queue, 14(1).
7. Merkel, D. (2014). *Docker: lightweight linux containers for consistent development and deployment*. Linux Journal.
8. Schwarzkopf, M., et al. (2013). *Omega: flexible, scalable schedulers for large compute clusters*. EuroSys.
9. WebSocket Protocol: RFC 6455
10. JSON Web Tokens: RFC 7519

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The core scheduling algorithm for CampusCompute is an **Adaptive Campus-Aware Resource Scheduler**. Unlike traditional cloud schedulers that optimize solely for resource availability, this algorithm incorporates institutional context.

Algorithm: Adaptive Campus Scheduler

Input : Container request R with CPU c , memory m , duration d

Output : Selected device D^* or queue placement

1. Filter Phase:

- $D_{candidates} \leftarrow \{d \in Devices \mid status = ONLINE\}$
- $D_{candidates} \leftarrow \{d \in D_{candidates} \mid dockerHealthy = true\}$
- $D_{candidates} \leftarrow \{d \in D_{candidates} \mid availableCPU \geq c\}$
- $D_{candidates} \leftarrow \{d \in D_{candidates} \mid availableRAM \geq m\}$
- $D_{candidates} \leftarrow \{d \in D_{candidates} \mid noConflictingReservation(d, d)\}$

2. Scoring Phase:

For each $d \in D_{candidates}$:

$$\begin{aligned} Score(d) = & w_1 \cdot \frac{availableCPU(d)}{totalCPU(d)} \\ & + w_2 \cdot \frac{availableRAM(d)}{totalRAM(d)} \\ & + w_3 \cdot (1 - currentLoad(d)) \\ & + w_4 \cdot reservationProximity(d, d) \\ & + w_5 \cdot reliability(d) \\ & + w_6 \cdot networkQuality(d) \end{aligned}$$

Where $w_1 + w_2 + \dots + w_6 = 1.0$

3. Selection Phase:

$$D^* \leftarrow \arg \max_{d \in D_{candidates}} Score(d)$$

4. Reservation Phase:

BEGIN TRANSACTION

Lock device resources for D^*

Verify availability still holds

Reserve c CPU and m RAM on D^*

COMMIT

5. Provisioning:

Send CREATE_CONTAINER command to agent on D^*

Novel Contribution: Reservation Proximity Function

The $reservationProximity(d, duration)$ function penalizes devices that have upcoming lab reservations within the requested duration window. This prevents scheduling long-running containers on machines that will soon be needed for classroom instruction.

$$reservationProximity(d, dur) = \begin{cases} 0.0 & \text{if reservation within } dur \\ 1.0 - \frac{1}{timeToNextReservation(d)} & \text{otherwise} \end{cases}$$

Data Structures:

- **Priority Queue:** For pending container requests
- **Hash Map:** Device ID to device state mapping
- **Interval Tree:** Efficient reservation conflict detection
- **Time Series Database:** Historical resource utilization

2.2 Characteristic of Data

CampusCompute manages several categories of data:

User Data:

- *Source:* Primary - University authentication system
- *Attributes:* User ID, email, role, department, quota allocation
- *Volume:* Estimated 5,000-10,000 users (students and faculty)
- *Growth:* Approximately 2,000 new users per academic year

Device Data:

- *Source:* Primary - Agent heartbeat messages

- *Attributes:* Device ID, lab location, CPU cores, RAM capacity, storage, status, Docker health
- *Volume:* 100-500 devices across campus labs
- *Frequency:* Heartbeat every 10 seconds

Resource Utilization Metrics:

- *Source:* Primary - Agent monitoring
- *Attributes:* Timestamp, device ID, CPU usage, RAM usage, network I/O, disk I/O
- *Sampling:* Every 10 seconds per device
- *Storage:* Time-series database with 30-day retention
- *Volume:* Approximately 4.3 million data points per day (500 devices $\times$ 8,640 samples/day)

Container Metadata:

- *Source:* Primary - Backend container management service
- *Attributes:* Container ID, user ID, device ID, image, resource allocation, status, creation time, expiration time
- *Volume:* Estimated 100-500 active containers at peak times

Audit Logs:

- *Source:* Primary - All system components
- *Attributes:* Timestamp, user ID, action type, resource type, result
- *Retention:* 1 year for compliance

Statistical Methods:

- *Moving Average:* For smoothing resource utilization metrics
- *Exponential Weighted Moving Average (EWMA):* For device reliability scoring
- *Percentile Analysis:* P95, P99 latency calculations for performance monitoring
- *Anomaly Detection:* Identifying unusual resource usage patterns

2.3 SWOT Analysis

Strengths:

- Novel academic-aware scheduling algorithm
- Utilizes existing infrastructure without additional hardware costs
- Browser-based access requires no client-side software installation
- Strong isolation through Docker containerization
- Scalable architecture supporting hundreds of devices
- Privacy-preserving design protecting student data

Weaknesses:

- Dependency on lab computer availability and network connectivity
- Requires Docker installation and agent deployment on all participating machines
- Limited to institutions with existing computer lab infrastructure
- Resource availability fluctuates with academic schedules
- Browser terminal performance inferior to native SSH clients

Opportunities:

- Expansion to multiple campuses within a university system
- Integration with learning management systems (Canvas, Moodle)
- Support for GPU-accelerated workloads for AI/ML courses
- Partnership with cloud providers for hybrid resource pooling
- Commercial licensing to other educational institutions
- Research opportunities in campus computing optimization

Threats:

- Commercial cloud providers offering education discounts (AWS Educate, Azure for Students)
- Security incidents could undermine institutional confidence
- Budget constraints limiting infrastructure maintenance
- Resistance from IT departments regarding security and management overhead
- Rapid technology changes requiring continuous system updates

2.4 Project Features

Core Features:

1. User Authentication and Authorization

- JWT-based secure authentication
- Role-based access control (Student, Faculty, Lab Admin, Super Admin)
- Integration with university SSO systems

2. Resource Discovery and Scheduling

- Automatic device registration and heartbeat monitoring
- Adaptive campus-aware scheduler
- Real-time resource availability tracking
- Queue management for pending requests

3. Container Lifecycle Management

- On-demand container provisioning
- Start, stop, restart, delete operations
- Automatic container expiration
- Resource limit enforcement (CPU, RAM, storage, PIDs)

4. Browser-Based Terminal

- Real-time terminal access via WebSocket
- xterm.js-powered terminal emulator
- Multiple concurrent sessions
- Session persistence and reconnection

5. Persistent Storage Management

- Docker volume creation and management
- File upload and download capabilities
- Storage quota enforcement
- Graceful data retention after container deletion

6. Lab Reservation System

- Schedule-aware resource allocation
- Lab-specific reservation creation

- Automatic scheduling conflict prevention
- Calendar integration

7. Quota Management

- Per-user and per-role resource limits
- CPU, memory, storage, and time quotas
- Automatic quota enforcement
- Admin quota adjustment interface

8. Monitoring and Analytics

- Real-time resource utilization dashboards
- Device health monitoring
- Container performance metrics
- Historical usage analysis
- Prometheus and Grafana integration

9. Administrative Controls

- Device enable/disable/drain operations
- User management
- System-wide configuration
- Audit log viewing

10. Security and Isolation

- Container security policies
- Network isolation
- mTLS agent authentication
- Ownership verification for all operations
- Comprehensive audit logging

2.5 User Classes and Characteristics

1. Students (Primary Users)

Characteristics:

- Basic to intermediate technical proficiency
- Require computing resources for coursework and projects

- Access system from various locations and devices
- Need simple, intuitive interfaces

Use Cases:

- Request containerized development environment
- Access terminal for command-line work
- Upload/download project files
- Monitor resource usage

2. Faculty (Secondary Users)

Characteristics:

- Variable technical proficiency
- Require resources for teaching and research
- May need to allocate resources to student groups
- Higher resource quotas than students

Use Cases:

- Provision environments for research projects
- Create shared resources for course assignments
- Monitor student resource usage (if authorized)

3. Lab Administrators

Characteristics:

- High technical proficiency
- Responsible for specific labs or departments
- Need granular control over assigned resources
- Monitor lab-specific metrics

Use Cases:

- Register and configure lab devices

- Create lab reservations
- Monitor lab utilization
- Manage device maintenance windows

4. System Administrators (Super Admins)

Characteristics:

- Expert technical proficiency
- Responsible for entire campus deployment
- Security and compliance oversight
- System configuration and policy management

Use Cases:

- Manage all devices across campus
- Configure system-wide policies and quotas
- Monitor system health and performance
- Review audit logs
- Handle security incidents

2.6 Design and Implementation Constraints

Hardware Constraints:

- *Lab Computers:* Must have sufficient resources (minimum 4 CPU cores, 8 GB RAM) to share resources while maintaining host functionality
- *Network:* Stable campus network connectivity required for agent-broker communication
- *Central Server:* Must handle 500+ concurrent WebSocket connections and database transactions
- *Storage:* Each device requires adequate storage for Docker images and container volumes

Software Constraints:

- *Docker Engine:* Version 24.0 or higher required on all agent machines

- *Java Runtime*: JDK 17 or higher for Spring Boot backend
- *Python*: Version 3.11 or higher for agent software
- *PostgreSQL*: Version 15 or higher for relational database
- *Modern Browser*: Chrome 90+, Firefox 88+, Safari 14+, or Edge 90+ for frontend

Security Constraints:

- All network communication must use TLS 1.3
- Agent-broker communication requires mutual TLS authentication
- Container must enforce strict resource limits and dropped capabilities
- No privileged containers permitted
- Audit logging mandatory for all administrative actions
- Password storage must use BCrypt or Argon2

Performance Constraints:

- Container startup time: ≤ 10 seconds (P95)
- Scheduling decision time: ≤ 2 seconds
- Terminal latency: ≤ 100 ms (P95)
- API response time: ≤ 500 ms (P95)
- Heartbeat interval: 10 seconds maximum

Technology Stack Requirements:

- *Frontend*: React 18, TypeScript, Vite, Tailwind CSS, xterm.js
- *Backend*: Spring Boot 3.2, Spring Security, Spring Data JPA
- *Agent*: Python 3.11, Docker SDK, websocket-client, psutil
- *Database*: PostgreSQL 15 with JSON support
- *Cache/Queue*: Redis 7
- *Reverse Proxy*: Nginx with WebSocket support

Design Standards:

- RESTful API design following OpenAPI 3.0 specification
- JSON Web Tokens (JWT) for stateless authentication
- WebSocket for real-time bidirectional communication
- Docker container security best practices
- Responsive web design for mobile and desktop

Operational Constraints:

- System must respect academic schedules and lab reservations
- Agent must not interfere with normal lab computer usage
- Graceful degradation when devices go offline
- Automatic recovery from network interruptions
- Zero-downtime deployment capability

2.7 Design Diagrams

Note: The following diagrams provide high-level views of system architecture and interactions. Detailed UML diagrams (Use Case, Class, Sequence, Activity, State, Data Flow, Deployment, and Collaboration diagrams) will be included in the final project documentation.

System Architecture Diagram:

[Placeholder: High-level three-layer architecture showing Frontend, Broker, and Agent layers with communication protocols]

Key Diagrams to be Developed:

1. **Use Case Diagram (Level 2):** Showing all user interactions including student container operations, admin device management, and system monitoring
2. **Class Diagram:** Core entities including User, Device, Container, Volume, Reservation, Quota, and their relationships
3. **Sequence Diagrams:**
 - Container creation workflow
 - Terminal session establishment
 - Device registration and heartbeat
 - Scheduler decision process
4. **Activity Diagrams:**

- Resource scheduling algorithm
 - Container lifecycle management
 - Authentication and authorization flow
5. **State Diagram:** Container state transitions (REQUESTED, ALLOCATING, PROVISIONING, RUNNING, STOPPED, DELETED)
 6. **Data Flow Diagram:** Information flow from user request through broker to agent and container
 7. **Deployment Diagram:** Physical distribution of components across central server and lab computers
 8. **Collaboration Diagram:** Object interactions during critical operations

2.8 Assumption and Dependencies

Assumptions:

1. Lab computers remain powered on during hours when resource sharing is enabled
2. Campus network provides stable connectivity with reasonable latency (< 50ms between buildings)
3. Docker Engine is properly configured and maintained on agent machines
4. Users have valid university credentials for authentication
5. Lab administrators create accurate reservation schedules
6. Students respect resource quotas and acceptable use policies
7. Operating systems on lab computers are kept up-to-date

Dependencies:

1. Infrastructure:

- Availability of central server for broker deployment
- Campus network infrastructure and firewall configurations
- IT department cooperation for agent deployment

2. External Software:

- Docker Engine stability and API compatibility
- PostgreSQL database reliability

- Nginx reverse proxy capabilities
- Browser WebSocket support

3. Third-Party Libraries:

- Spring Boot framework maintenance and security updates
- React and npm package ecosystem stability
- Python Docker SDK continued support
- xterm.js library development

4. Institutional:

- University authentication system availability
- Compliance with institutional IT security policies
- Academic schedule data accessibility
- Support from faculty and administration

5. Development:

- Team member availability throughout project timeline
- Access to development and testing infrastructure
- Mentor guidance and approval at key milestones

3 SYSTEM REQUIREMENTS

3.1 User Interface

Student Portal Components:

1. Login Page:

- Email and password input fields
- "Remember Me" option
- "Forgot Password" link
- University SSO integration button

2. Dashboard:

- Campus resource availability overview (CPU, RAM, storage)
- Personal quota status visualization
- Active container list with status indicators
- Quick action buttons (Create Environment, View All)

3. Container Management:

- Container creation form with image selection, resource sliders, duration picker
- Container list table with status, resources, location, actions
- Container detail view with metrics, logs, terminal access
- Control buttons (Start, Stop, Delete, Open Terminal)

4. Browser Terminal:

- Full-screen xterm.js terminal emulator
- Theme customization (dark/light mode)
- Font size adjustment
- Copy/paste functionality
- Session reconnection handling

5. File Manager:

- Directory tree navigation
- File upload with drag-and-drop
- File download
- Delete confirmation dialogs

- Storage usage indicator

Admin Portal Components:

1. Dashboard:

- Campus-wide resource utilization charts
- Device status overview (online, offline, maintenance)
- Active container count and distribution
- Recent system events feed

2. Device Management:

- Device list with status, location, resources, actions
- Device enrollment interface with token generation
- Device detail view with real-time metrics
- Control buttons (Enable, Disable, Drain, Maintenance Mode)

3. Lab Management:

- Lab list with department, building, device count
- Lab creation and configuration forms
- Lab-level resource allocation policies

4. Reservation Management:

- Calendar view of lab reservations
- Reservation creation form with lab selection, time range, recurrence
- Conflict detection and warnings
- Edit and delete capabilities

5. User Management:

- User list with search and filter
- Role assignment interface
- Quota configuration per user or role
- User activity history

6. Monitoring:

- Real-time resource utilization graphs
- Container distribution heatmaps

- Performance metrics (P95, P99 latencies)
- Grafana dashboard integration

7. Audit Logs:

- Searchable log viewer with date range filter
- Action type filtering
- User and resource filtering
- Export functionality

UI Requirements:

- Responsive design supporting desktop (1920x1080), tablet (768x1024), and mobile (375x667) viewports
- WCAG 2.1 Level AA accessibility compliance
- Loading states and progress indicators for async operations
- Error messages with clear remediation instructions
- Confirmation dialogs for destructive actions
- Real-time status updates via WebSocket
- Dark mode support

3.2 Software Interface

Frontend to Backend Interface:

- *Protocol:* HTTPS REST API and WebSocket (WSS)
- *Authentication:* Bearer token (JWT) in Authorization header
- *Data Format:* JSON for request/response bodies
- *API Versioning:* URL path versioning (e.g., /api/v1/)
- *Error Handling:* Standard HTTP status codes with JSON error objects

Backend to Agent Interface:

- *Protocol:* WebSocket over mutual TLS (mTLS)
- *Authentication:* X.509 certificates (device certificates)

- *Message Format*: JSON with command type and payload
- *Message Types*:
 - HEARTBEAT: Agent → Broker
 - CREATE_CONTAINER: Broker → Agent
 - START_CONTAINER: Broker → Agent
 - STOP_CONTAINER: Broker → Agent
 - DELETE_CONTAINER: Broker → Agent
 - EXEC_TERMINAL: Broker → Agent
 - STATUS_UPDATE: Agent → Broker
- *Connection*: Persistent WebSocket initiated by agent (outbound)
- *Reconnection*: Exponential backoff (1s, 2s, 4s, ..., max 60s)

Backend to Docker Interface (via Agent):

- *Library*: Docker SDK for Python (docker-py)
- *Connection*: Unix socket (/var/run/docker.sock) or TCP with TLS
- *Operations*:
 - Container create, start, stop, remove
 - Volume create, remove
 - Resource inspection and statistics
 - Exec session creation for terminal access

Backend to Database Interface:

- *ORM*: Hibernate via Spring Data JPA
- *Connection Pool*: HikariCP (default Spring Boot)
- *Transaction Management*: Spring @Transactional annotations
- *Migration*: Flyway or Liquibase for schema versioning

Backend to Cache Interface:

- *Client*: Spring Data Redis
- *Protocol*: RESP (Redis Serialization Protocol)

- *Use Cases:*
 - JWT token blacklist for logout
 - Temporary resource allocation state
 - WebSocket session routing information
 - Rate limiting counters

External Authentication Interface:

- *Protocol:* OAuth 2.0 / SAML 2.0 (for university SSO)
- *Fallback:* Local authentication with BCrypt password hashing

3.3 Database Interface

Database Management System: PostgreSQL 15

Connection Configuration:

- *Driver:* JDBC PostgreSQL Driver (org.postgresql:postgresql)
- *Connection String:* jdbc:postgresql://localhost:5432/campuscompute
- *Pool Size:* 20 connections (adjustable based on load)
- *Connection Timeout:* 30 seconds

Core Database Tables:

1. **users:** User accounts with authentication credentials
2. **departments:** Organizational units within university
3. **labs:** Physical computer labs with metadata
4. **devices:** Individual lab computers registered with system
5. **device_resources:** Resource capacity for each device
6. **containers:** Active and historical container records
7. **volumes:** Persistent storage volumes
8. **resource_allocations:** CPU, memory, storage assigned to containers
9. **reservations:** Lab reservation schedules

10. **quotas:** Resource limits per user or role
11. **heartbeats:** Device health status (time-series, partitioned)
12. **resource_usage:** Container utilization metrics (time-series)
13. **audit_logs:** System action audit trail

Database Features Utilized:

- *JSON/JSONB:* For flexible metadata storage
- *Indexes:* B-tree and GIN indexes for query optimization
- *Partitioning:* Table partitioning for time-series data
- *Constraints:* Foreign keys, unique constraints, check constraints
- *Triggers:* For automatic timestamp updates
- *Views:* Materialized views for complex analytics queries

Backup and Recovery:

- *Strategy:* Daily automated backups with 30-day retention
- *Method:* pg_dump for logical backups
- *Point-in-Time Recovery:* WAL archiving enabled

3.4 Protocols

HTTPS (HTTP/1.1 over TLS 1.3):

- *Purpose:* Secure REST API communication between frontend and backend
- *Port:* 443 (standard HTTPS)
- *Cipher Suites:* TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256
- *Certificate:* Let's Encrypt or institutional CA-signed certificate
- *HSTS:* Strict-Transport-Security header enforced

WebSocket Secure (WSS):

- *Purpose:* Real-time bidirectional communication for terminal sessions and agent connections

- *Protocol:* RFC 6455 WebSocket over TLS 1.3
- *Handshake:* HTTP Upgrade with Sec-WebSocket-Key validation
- *Frames:* Text frames for JSON messages, Binary frames for terminal data
- *Keep-Alive:* Ping/Pong frames every 30 seconds
- *Closure:* Graceful close with 1000 (Normal Closure) or 1001 (Going Away) codes

Mutual TLS (mTLS):

- *Purpose:* Agent authentication to broker
- *Mechanism:* Both server and client present X.509 certificates
- *Server Certificate:* Identifies broker to agents
- *Client Certificate:* Identifies specific agent device
- *Verification:* Certificate chain validation against trusted CA
- *Revocation:* CRL or OCSP for compromised certificates

JWT (JSON Web Token - RFC 7519):

- *Purpose:* Stateless user authentication
- *Algorithm:* HS256 (HMAC with SHA-256) or RS256 (RSA with SHA-256)
- *Claims:* sub (user ID), role, department, iat (issued at), exp (expiration)
- *Expiration:* 24 hours for access tokens
- *Refresh:* Refresh tokens with 7-day expiration
- *Storage:* HttpOnly, Secure cookies (preferred) or localStorage with XSS mitigation

Docker API:

- *Purpose:* Container and volume management on agent machines
- *Protocol:* RESTful HTTP API (Engine API v1.43)
- *Transport:* Unix socket (/var/run/docker.sock) or TCP with TLS
- *Authentication:* Client certificate authentication for remote access

Security Considerations:

- All network communication encrypted with TLS 1.3
- No plaintext transmission of credentials
- Certificate pinning for agent-broker connections
- Rate limiting on API endpoints (100 requests/minute per user)
- CORS policy restricting frontend origin
- WebSocket origin validation

Data Transfer:

- *File Upload:* Multipart form data with size limit (100 MB per file)
- *File Download:* Streaming response with Content-Disposition header
- *Terminal Data:* UTF-8 encoded text in WebSocket text frames
- *Compression:* Gzip compression for HTTP responses > 1KB

Synchronization:

- *Heartbeat:* Agent sends status every 10 seconds
- *State Reconciliation:* On agent reconnection, broker queries actual container states
- *Clock Synchronization:* NTP required on all devices for accurate scheduling
- *Event Ordering:* Sequence numbers in WebSocket messages

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

Response Time Requirements:

- *API Response:* 95% of requests $\leq$ 500ms, 99% $\leq$ 1s
- *Container Startup:* P95 $\leq$ 10s, P99 $\leq$ 15s
- *Scheduling Decision:* $\leq$ 2s for 95% of requests
- *Terminal Latency:* $\leq$ 100ms for keystroke echo (P95)
- *Page Load:* Initial load $\leq$ 3s, subsequent navigation $\leq$ 1s
- *WebSocket Connection:* Establishment $\leq$ 500ms

Throughput Requirements:

- *Concurrent Users:* Support 500 simultaneous active users
- *Container Requests:* Process 50 container creation requests/minute
- *Terminal Sessions:* Support 200 concurrent terminal sessions
- *Heartbeats:* Process 50 heartbeat messages/second (500 devices $\times$ 10s interval)
- *File Transfers:* Support 20 concurrent file uploads/downloads

Capacity Requirements:

- *Devices:* Scale to 500 registered lab computers
- *Users:* Support 10,000 user accounts
- *Containers:* Manage up to 1,000 concurrent containers
- *Database:* 100 million rows (with partitioning for time-series data)
- *Storage:* 10 TB aggregate student workspace storage

Resource Utilization:

- *Backend Memory:* $\leq$ 4 GB under normal load
- *Backend CPU:* $\leq$ 50% of 4-core processor under normal load

- *Agent Memory*: ≤ 200 MB per agent
- *Agent CPU*: ≤ 5% of host CPU when idle
- *Database*: Query execution time ≤ 100ms for 95% of queries

Scalability:

- *Horizontal Scaling*: Backend should support load balancing across multiple instances
- *Database*: Read replicas for analytics queries
- *Stateless Design*: No session affinity required for API servers
- *Resource Limits*: Graceful degradation when approaching capacity

4.2 Security Requirements

Authentication Requirements:

- Strong password policy (minimum 12 characters, mixed case, numbers, symbols)
- Multi-factor authentication (MFA) support for administrative accounts
- Account lockout after 5 failed login attempts
- Password reset via email verification
- Integration with university Single Sign-On (SSO) system
- Device certificate-based authentication for agents

Authorization Requirements:

- Role-Based Access Control (RBAC) with four roles: Student, Faculty, Lab Admin, Super Admin
- Ownership verification for all resource operations (containers, volumes)
- Principle of least privilege enforced throughout system
- Administrative actions require elevated privileges
- API endpoint protection with role-based guards

Data Protection:

- All passwords hashed using BCrypt (cost factor 12) or Argon2
- Sensitive data encrypted at rest in database
- TLS 1.3 for all network communication
- No logging of passwords or authentication tokens
- Automatic session termination after 24 hours

Container Security:

- No privileged containers allowed
- Docker socket never mounted in student containers
- Dropped Linux capabilities (ALL except CHOWN, SETUID, SETGID)
- Resource limits enforced (CPU, memory, PIDs, storage)
- Read-only root filesystem where possible
- Security option: no-new-privileges flag
- Network isolation between containers
- Regular base image security scanning

Audit and Monitoring:

- Comprehensive audit logging of administrative actions
- Login/logout events logged with IP addresses
- Container creation, modification, deletion logged
- Failed authentication attempts tracked
- Anomalous resource usage detection
- Security incident alerting
- Log retention for 1 year minimum

Network Security:

- Firewall rules limiting exposed ports
- Rate limiting on public API endpoints

- DDoS protection at Nginx layer
- CORS policy restricting frontend origins
- Content Security Policy (CSP) headers
- X-Frame-Options to prevent clickjacking

Compliance:

- FERPA compliance for student educational records
- Institutional data governance policies
- Regular security audits and penetration testing
- Incident response plan documentation

4.3 Software Quality Attributes

Reliability:

- *Availability*: 99% uptime excluding scheduled maintenance
- *MTBF*: Mean Time Between Failures $\geq$ 720 hours
- *MTTR*: Mean Time To Recovery $\leq$ 1 hour
- *Fault Tolerance*: Graceful degradation when devices go offline
- *Data Integrity*: Database transactions ensure consistency
- *Error Recovery*: Automatic reconnection for agent-broker communication

Availability:

- System accessible 24/7 except scheduled maintenance windows
- Scheduled maintenance limited to 4 hours/month during low-usage periods
- Redundant components where practical (database, cache)
- Health check endpoints for monitoring
- Automated restart on component failure

Maintainability:

- *Modularity*: Clear separation of concerns (auth, scheduling, containers)
- *Documentation*: Comprehensive code comments and API documentation
- *Code Standards*: Consistent formatting and naming conventions
- *Version Control*: Git with feature branch workflow
- *Testing*: Minimum 70% code coverage
- *Logging*: Structured logging with log levels
- *Configuration*: Externalized configuration files

Portability:

- *Cross-Platform*: Backend runs on Linux and Windows Server
- *Browser Support*: Chrome, Firefox, Safari, Edge (latest 2 versions)
- *Containerized Deployment*: Docker Compose for easy setup
- *Database Independence*: JPA abstraction (though PostgreSQL recommended)
- *Cloud-Agnostic*: Deployable on-premises or cloud providers

Usability:

- *Learnability*: New users can create container within 5 minutes
- *Efficiency*: Power users can perform common tasks quickly
- *Memorability*: Infrequent users can remember how to use system
- *Error Prevention*: Confirmation dialogs for destructive actions
- *Error Recovery*: Clear error messages with remediation steps
- *Satisfaction*: Intuitive interface following modern design patterns
- *Accessibility*: WCAG 2.1 Level AA compliance

Testability:

- Unit tests for service layer logic
- Integration tests for API endpoints
- End-to-end tests for critical user workflows

- Performance tests for load scenarios
- Security tests for vulnerability scanning
- Continuous Integration (CI) pipeline
- Test data generation utilities

Flexibility:

- Configurable scheduling weights
- Pluggable authentication providers
- Extensible quota policies
- Customizable container images
- Configurable resource limits

Reusability:

- RESTful API enables third-party integration
- Modular architecture supports component reuse
- Library abstractions for Docker, WebSocket, authentication
- Open API specification for client generation

Robustness:

- Input validation on all API endpoints
- SQL injection prevention via parameterized queries
- XSS prevention via output encoding
- CSRF protection with tokens
- Resource exhaustion prevention (rate limiting, quotas)
- Graceful handling of unexpected inputs

Correctness:

- Adherence to specifications throughout implementation

- Accurate resource accounting and quota enforcement
- Consistent state management across distributed components
- Validation of all user inputs
- Comprehensive testing of business logic

Interoperability:

- Standard REST API following OpenAPI specification
- WebSocket protocol compliance (RFC 6455)
- Docker API compatibility
- OAuth 2.0 / SAML 2.0 for SSO integration
- Prometheus metrics format for monitoring integration
- Standard SQL database interface

5 OTHER REQUIREMENTS

Legal and Licensing:

- Compliance with university acceptable use policies
- Open source license compatibility (Apache 2.0, MIT)
- Third-party library license review
- Terms of service and privacy policy for users
- Data retention and deletion policies

Documentation Requirements:

- User manual for students
- Administrator guide for IT staff
- API reference documentation
- Architecture and design documentation
- Deployment and installation guide
- Security policies and procedures
- Troubleshooting and FAQ

Training Requirements:

- Training sessions for system administrators
- Video tutorials for student onboarding
- Documentation for faculty integration
- Help desk support materials

Operational Requirements:

- Automated backup procedures
- System monitoring and alerting
- Log aggregation and analysis

- Capacity planning and forecasting
- Disaster recovery procedures
- Regular security updates

Research and Evaluation:

- Experimental design for scheduler evaluation
- Metrics collection for performance analysis
- User satisfaction surveys
- Comparative analysis with traditional approaches
- Publication of research findings

Future Enhancements:

- GPU resource sharing for AI/ML workloads
- Multi-campus federation
- Integration with learning management systems
- Mobile application development
- Advanced container orchestration (Kubernetes)
- Hybrid cloud bursting to public cloud

APPENDIX A: GLOSSARY

Agent: Python software deployed on lab computers that manages Docker containers and communicates resource availability to the broker.

Broker: Central Spring Boot application that orchestrates resource allocation, scheduling, and container management across campus.

Container: Isolated Docker environment provisioned for a user with dedicated resource allocation.

Device: A physical lab computer registered with CampusCompute that can host containers.

Drain: Administrative action that prevents new container allocation on a device while allowing existing containers to complete.

Heartbeat: Periodic status message sent by agent to broker indicating device health and resource availability.

JWT (JSON Web Token): Stateless authentication token containing user identity and authorization claims.

Lab: Physical computer laboratory within a department containing multiple devices.

mTLS (Mutual TLS): Authentication protocol where both client and server present certificates to establish identity.

Quota: Resource limit assigned to a user or role defining maximum CPU, memory, storage, and container count.

Reservation: Scheduled time period during which a lab is unavailable for container allocation due to classes or maintenance.

Scheduler: Component responsible for selecting optimal device for container placement based on resource availability and institutional context.

Volume: Persistent Docker storage volume mounted in container for user workspace data.

WebSocket: Protocol enabling full-duplex communication between client and server over a single TCP connection.

xterm.js: JavaScript library providing terminal emulation in web browsers.

RBAC (Role-Based Access Control): Authorization model granting permissions based on user roles rather than individual identities.

BCrypt: Password hashing algorithm designed to be computationally expensive to resist brute-force attacks.

Docker: Platform for developing, shipping, and running applications in isolated containers.

Spring Boot: Java framework for building production-ready applications with minimal configuration.

React: JavaScript library for building user interfaces with component-based architecture.

PostgreSQL: Open-source relational database management system.

Redis: In-memory data structure store used for caching and message queuing.

Prometheus: Open-source monitoring and alerting toolkit.

Grafana: Open-source analytics and visualization platform.

REST (Representational State Transfer): Architectural style for distributed systems based on stateless client-server communication.

API (Application Programming Interface): Set of protocols and tools for building software applications.

SSO (Single Sign-On): Authentication scheme allowing users to log in once and access multiple systems.

FERPA: Family Educational Rights and Privacy Act - U.S. federal law protecting student education records.

WCAG (Web Content Accessibility Guidelines): Set of recommendations for making web content more accessible to people with disabilities.

ORM (Object-Relational Mapping): Programming technique for converting data between incompatible type systems (objects and relational databases).

JPA (Java Persistence API): Java specification for accessing, persisting, and managing data between Java objects and relational databases.

CORS (Cross-Origin Resource Sharing): Mechanism allowing web applications to access resources from different origins.

CSP (Content Security Policy): Security standard helping prevent cross-site scripting and other code injection attacks.

APPENDIX B: ANALYSIS MODEL

Resource Utilization Model:

The system tracks resource utilization at multiple levels:

$$U_{campus} = \frac{\sum_{i=1}^n (CPU_{allocated,i} + RAM_{allocated,i})}{\sum_{i=1}^n (CPU_{total,i} + RAM_{total,i})}$$
$$U_{device} = \frac{CPU_{allocated} + RAM_{allocated}}{CPU_{total} + RAM_{total}}$$

Where n is the number of devices and utilization is measured as percentage of available resources.

Scheduling Score Model:

The adaptive scheduler uses a weighted scoring function:

$$Score(d) = \sum_{i=1}^6 w_i \cdot f_i(d)$$

Where:

- $f_1(d)$ = CPU availability factor
- $f_2(d)$ = RAM availability factor
- $f_3(d)$ = Current load factor (inverse)
- $f_4(d)$ = Reservation proximity factor
- $f_5(d)$ = Reliability factor (EWMA)
- $f_6(d)$ = Network quality factor
- $\sum w_i = 1.0$

Quota Enforcement Model:

For each user u and resource type r :

$$allocated_r(u) = \sum_{c \in containers(u)} resource_r(c)$$
$$allowed(u, r) \Leftrightarrow allocated_r(u) + requested_r \leq quota_r(u)$$

Reliability Model:

Device reliability is calculated using Exponential Weighted Moving Average (EWMA):

$$reliability_t = \alpha \cdot success_t + (1 - \alpha) \cdot reliability_{t-1}$$

Where $\alpha = 0.3$ (smoothing factor) and $success_t \in \{0, 1\}$ based on operation outcomes.

Performance Model:

Expected container startup time:

$$T_{startup} = T_{scheduling} + T_{provisioning} + T_{docker_create} + T_{docker_start}$$

Target: $T_{startup} < 10s$ for P95.

Capacity Planning Model:

Maximum concurrent containers:

$$C_{max} = \frac{\sum_{i=1}^n CPU_{total,i}}{CPU_{avg_per_container}}$$

With safety margin: $C_{target} = 0.85 \times C_{max}$

Cost-Benefit Analysis:

- *Infrastructure Cost Avoidance:* \$500/student for personal hardware vs. \$50/student for CampusCompute deployment
- *Utilization Improvement:* 18% $\rightarrow$ 60% average lab utilization
- *ROI Timeline:* Expected payback period of 18 months

APPENDIX C: ISSUES LIST

This section maintains a dynamic list of open requirements issues to be resolved during project execution.

ID	Issue Description	Impact	Status
1	Determine optimal Docker image approval workflow for student-requested custom images	Security vs. flexibility tradeoff	Open
2	Define specific thresholds for adaptive scheduler weight adjustment based on time-of-day patterns	Scheduling effectiveness	Open
3	Clarify GPU resource sharing requirements and feasibility for future phases	Feature scope	Open
4	Establish maximum container duration limits balancing student needs and fair resource access	Policy definition	Open
5	Define disaster recovery RTO and RPO targets for database and persistent volumes	Operational requirements	Open
6	Determine integration requirements with university's existing authentication system (LDAP, OAuth, SAML)	Technical integration	Open
7	Clarify network policies for container internet access (unrestricted, whitelist, or isolated)	Security policy	Open
8	Define metrics collection strategy: push vs. pull model for Prometheus integration	Monitoring architecture	Open
9	Establish testing environment requirements: dedicated test lab or shared development resources	Testing infrastructure	Open
10	Define automatic container cleanup policy for abandoned or expired containers	Resource management	Open

ID	Issue Description	Impact	Status
11	Clarify performance monitoring dashboard requirements for students vs. administrators	UI requirements	Open
12	Determine approach for handling agent software updates across hundreds of devices	Deployment strategy	Open
13	Define scheduling behavior when no devices meet minimum requirements: queue vs. rejection	User experience	Open
14	Establish data retention policies for audit logs, metrics, and deleted container volumes	Compliance	Open
15	Clarify whether file sharing between users is required or explicitly prohibited	Feature scope	Open
16	Define notification system requirements: email, in-app, push notifications for container status	Communication channels	Open
17	Determine multi-tenancy model if system will serve multiple departments with isolation requirements	Architecture decision	Open
18	Establish criteria for measuring project success beyond technical metrics	Evaluation framework	Open

Note: This issues list will be updated throughout the project lifecycle as requirements are clarified and new issues emerge.